Lecture 5 — Binary Search & Binary Search on the Answer

> Year Curriculum · Prerequisites: Weeks 2–4 (STL, complexity, sorting & greedy) · Estimated time: 2 hours lecture + 4 hours practice

1. Overview

Binary search is the simplest logarithmic algorithm. In each step, look at the middle of the remaining range and discard the half that cannot contain the answer. A range of size N shrinks to one element in $\lceil \log_2 N \rceil$ steps: 30 steps for 10^9 and 60 for 10^18.

Binary search is famous for being easy to describe and hard to code correctly. Off-by-one errors and infinite loops are common. This lecture gives one **invariant-based template** that avoids both, and then applies it in increasingly abstract settings:

1. Searching a sorted array, and the STL's `lower_bound`/`upper_bound`.
2. The general view: finding the **boundary of a monotone predicate**.
3. **Binary search on the answer**, which turns an optimization problem into a sequence of yes/no feasibility checks.
4. "Minimize the maximum" and "maximize the minimum" problems.
5. Binary search over real numbers.

2. The monotone-predicate view

Let P be a predicate on integers in a range [L, R] that is **monotone**: false for some prefix and then true for the rest of the range:

```
x:    L ........................... R
P(x):  F F F F F F F T T T T T T T T T T
                    ^ boundary
```

Every binary search finds this boundary. Examples:

- In a sorted array a, P(i) = (a[i] ≥ x). The boundary is `lower_bound(x)`.
- P(T) = "the machines can produce t items within time T". The boundary is the minimum time.
- P(S) = "the array can be split into ≤ k parts each with sum ≤ S". The boundary is the minimal possible maximum part.

If P is not monotone, binary search is meaningless and returns an arbitrary point. **Always argue monotonicity explicitly.**

3. The invariant template

Maintain two variables with this invariant:

> **P(lo) is false and P(hi) is true.**

They may be "virtual" positions outside the real range. For example, set lo = L − 1 (pretend P is false there) and hi = R + 1 (pretend P is true there, meaning "no true value exists").

```
long long lo = L - 1, hi = R + 1;     // P(lo) = false, P(hi) = true
while (hi - lo > 1) {
    long long mid = lo + (hi - lo) / 2;
    if (P(mid)) hi = mid;
    else        lo = mid;
}
// hi = first x with P(x) true  (or R + 1 if none)
// lo = last  x with P(x) false (or L - 1 if none)
```

Why it is correct:

- **Invariant maintained.** mid is strictly between lo and hi, because hi − lo ≥ 2. If P(mid) is true, setting hi = mid keeps "P(hi) true", and likewise for lo.
- **Termination.** The gap hi − lo strictly decreases each step, because mid ≠ lo and mid ≠ hi.
- **Result.** When the loop ends, hi = lo + 1 with P(lo) false and P(hi) true. That pair is the boundary.
- **P is never evaluated at the virtual ends**, so they need not be valid inputs.

Because `mid` is strictly inside the range, the template never needs `mid + 1` or `mid − 1`, never needs `while (lo <= hi)`, and never loops forever. For "last true" (the predicate is true…true then false…false), swap the roles: the invariant becomes P(lo) true and P(hi) false, and the answer is lo.

`lo + (hi − lo) / 2` avoids the overflow that `(lo + hi) / 2` can cause near 2^63. When lo may be negative, it also rounds consistently.

3.1 Trace

Array a = [1, 3, 4, 7, 9, 12, 15, 20], with P(i) = (a[i] ≥ 8):

lo	hi	mid	a[mid]	P	action
−1	8	3	7	F	lo = 3
3	8	5	12	T	hi = 5

lo	hi	mid	a[mid]	P	action
3	5	4	9	T	hi = 4
3	4	—	—	—	stop

Answer: hi = 4, where a[4] = 9 is the first element ≥ 8.

4. The STL functions

On a sorted random-access range:

- `lower_bound(first, last, x)`: iterator to the first element **≥ x**.
- `upper_bound(first, last, x)`: iterator to the first element **> x**.
- `binary_search(first, last, x)`: whether x is present.
- `equal_range(first, last, x)`: the pair (lower_bound, upper_bound).

Derived quantities, for a sorted vector `a` of size n:

Quantity	Expression
number of elements < x	`lower_bound(all(a), x) - a.begin()`
number of elements ≤ x	`upper_bound(all(a), x) - a.begin()`
number equal to x	`upper - lower`
number in [l, r]	`upper_bound(r) - lower_bound(l)`
largest element ≤ x	the element before `upper_bound(x)`, if any
smallest element ≥ x	`*lower_bound(x)`, if not `end()`

All accept a comparator as the fourth argument, which must be the same ordering the range is sorted by. For example, on a descending array use `lower_bound(all(a), x, greater<int>())`. `set` and `map` have **member** versions, `s.lower_bound(x)`. The free function on a set is O(n).

Example — CF 474B Worms. Pile i contains labels pref[i−1]+1 … pref[i], where pref holds the prefix sums. For a label q, the answer is the first i with pref[i] ≥ q, which is a `lower_bound` on the prefix-sum array. With m queries the total is O((n + m) log n).

Example — CF 706B Interesting drink. Shop prices x_i. For each day's budget m, count the shops with price ≤ m. Sort once, and answer each query with `upper_bound`.

5. Binary search on the answer

Many problems ask for the **minimum value of something such that a condition holds**, or the maximum. Computing the optimum directly is hard, but **checking** a candidate is easy. If feasibility is monotone in the candidate, binary search over the candidate values:

1. Define `ok(x)`.
2. Prove that ok(x) ⇒ ok(x + 1) (for minimization).
3. Pick lo with ok(lo) false and hi with ok(hi) true. Make sure hi is *really* feasible.
4. Run the template. The answer is hi.

Cost: O(log(hi − lo)) calls to `ok`. With ranges up to 10^18 that is about 60 calls, so an O(n) check is fine for n up to about 10^6.

5.1 CSES 1620 — Factory Machines

Machine i produces one item every k_i seconds, and all machines work in parallel. Find the minimum time to produce t items.

- ok(T) = Σ ⌊T / k_i⌋ ≥ t. This is monotone, because more time never produces fewer items.
- lo = 0, since nothing is produced at time 0 and t ≥ 1. hi = min(k)·t, because the fastest machine alone produces t items in that time. hi ≤ 10^9 · 10^9 = 10^18, which fits in `long long`.
- **Overflow inside the check.** Σ ⌊T / k_i⌋ can reach 2·10^5 · 10^18, which overflows. Stop summing as soon as the total reaches t.

Complexity: O(n log(10^18)) ≈ 1.2·10^7 operations.

5.2 CSES 1085 — Array Division (minimize the maximum)

Divide an array of positive integers into k contiguous subarrays, minimizing the largest subarray sum.

- ok(S) = "the array can be divided into at most k parts, each with sum ≤ S". (At most k is enough: a part of size ≥ 2 can always be split further without increasing any sum, as long as k ≤ n.)
- **Greedy check.** Scan left to right, extending the current part while its sum stays ≤ S, and start a new part otherwise. This uses the minimum possible number of parts for a given S: a simple exchange argument shows that ending a part as late as possible never hurts.
- Monotone: if S works, any larger S works.
- lo = max(a) − 1, which is infeasible because the largest element cannot fit in any part. hi = Σa, which is feasible with one part.

Every `mid` evaluated satisfies mid ≥ max(a), so the check never meets an element larger than S.

5.3 Maximize the minimum

The classic "aggressive cows" problem: place c cows in stalls at positions x_1 < … < x_n to maximize the minimum distance between any two cows.

- ok(d) = "we can place c cows with all pairwise gaps ≥ d". Greedy: put a cow in the first stall, then each next cow in the first stall at distance ≥ d from the previous cow.
- Now ok is **true then false** as d grows, and we want the last true. Use the invariant ok(lo) true, ok(hi) false: lo = 0, which is trivially fine with all gaps ≥ 0, and hi = (x_n − x_1) + 1. The answer is lo.

5.4 CF 1201C — Maximum Median

An odd-length array and k unit increments. Maximize the median. Sort. ok(m) = "the cost to raise every element from the median position to the end up to at least m is ≤ k", that is, Σ_{i ≥ n/2} max(0, m − a_i) ≤ k. This is monotone decreasing in m, so binary search for the largest feasible m. The cost can reach about 2·10^5 · 2·10^9, so use `long long` and stop early once it exceeds k.

5.5 Recognizing the pattern

Signals that suggest binary search on the answer:

- "minimum possible maximum…", "maximum possible minimum…";
- "minimum time / speed / capacity such that…";
- a check is easy, often a greedy or a simple simulation, while direct optimization is not;
- the answer range is huge (up to 10^9 or 10^18), but O(log) checks are affordable.

6. Binary search on real numbers

When the answer is real-valued, such as a distance or a rate, run a **fixed number of iterations** rather than looping until `hi - lo < eps`:

```
double lo = 0, hi = 1e9;
for (int it = 0; it < 100; it++) {
    double mid = (lo + hi) / 2;
    if (ok(mid)) hi = mid; else lo = mid;
}
```

With large values, adjacent doubles can be farther apart than `eps`, so an `eps` loop may never terminate. One hundred halvings reduce any range below the precision of a `double`. If each check is expensive, 50–60 iterations usually suffice for 1e-9 precision relative to the range.

Classic application: **maximize an average**. "Is there a subarray of length ≥ L with average ≥ x?" becomes, after subtracting x from every element, "is there a subarray of length ≥ L with sum ≥ 0?", which prefix sums (Week 7) answer in O(n).

Integer square roots. `sqrt(n)` for n near 10^18 can be off by one because doubles have 53-bit mantissas. Either correct the floating-point estimate with small loops (as in the slide), or binary search with the predicate r ≤ n / r, which avoids computing r·r.

7. Reference implementations

7.1 Factory Machines (CSES 1620)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long t;
    cin >> n >> t;
    vector<long long> k(n);
    for (auto &x : k) cin >> x;

    auto ok = [&](long long T) {
        long long made = 0;
        for (long long x : k) {
            made += T / x;
            if (made >= t) return true;     // early exit prevents overflow
        }
        return false;
    };

    long long lo = 0;                                   // ok(0) is false
    long long hi = *min_element(k.begin(), k.end()) * t; // ok(hi) is true
    while (hi - lo > 1) {
        long long mid = lo + (hi - lo) / 2;
        if (ok(mid)) hi = mid;
        else lo = mid;
    }
    cout << hi << '\n';
    return 0;
}
```

Sample: `3 7` / `3 2 5` → `8`.

7.2 Array Division (CSES 1085)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
```

```
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;

    auto ok = [&](long long S) {            // requires S >= max(a)
        int parts = 1;
        long long cur = 0;
        for (long long x : a) {
            if (cur + x > S) { parts++; cur = 0; }
            cur += x;
        }
        return parts <= k;
    };

    long long lo = *max_element(a.begin(), a.end()) - 1;   // infeasible
    long long hi = accumulate(a.begin(), a.end(), 0LL);    // feasible
    while (hi - lo > 1) {
        long long mid = lo + (hi - lo) / 2;
        if (ok(mid)) hi = mid;
        else lo = mid;
    }
    cout << hi << '\n';
    return 0;
}
```

Sample: `5 3` / `2 4 7 3 5` → `8`.

7.3 Aggressive cows (maximize the minimum gap)

```
#include <bits/stdc++.h>
using namespace std;

// Input: n c, then n stall positions. Output: largest minimum gap.
int main() {
    int n, c;
    cin >> n >> c;
    vector<long long> x(n);
    for (auto &v : x) cin >> v;
    sort(x.begin(), x.end());

    auto ok = [&](long long d) {            // can we place c cows with gaps >= d?
        int placed = 1;
        long long last = x[0];
        for (int i = 1; i < n && placed < c; i++)
            if (x[i] - last >= d) { placed++; last = x[i]; }
        return placed >= c;
    };

    long long lo = 0, hi = x[n - 1] - x[0] + 1;   // ok(lo) true, ok(hi) false
    while (hi - lo > 1) {
        long long mid = lo + (hi - lo) / 2;
```

```
        if (ok(mid)) lo = mid;
        else hi = mid;
    }
    cout << lo << '\n';
    return 0;
}
```

Sample: `5 3` / `1 2 8 4 9` → `3`. Cows go at 1, 4, and 8 (or 9).

(`hi` is infeasible whenever c ≥ 2, since no two stalls are that far apart. With c = 1 the "minimum gap" is undefined. Problems guarantee c ≥ 2.)

8. Common mistakes

1. **Non-monotone predicate.** Before coding, write down why ok(x) implies ok(x+1).
2. **Bad bounds.** hi must truly be feasible and lo truly infeasible, or the "answer" may be a value that was never checked. If unsure, check `ok(hi)` explicitly and handle "impossible".
3. **Overflow in `mid`.** Use `lo + (hi - lo) / 2`.
4. **Overflow inside the check.** Sums of quotients, products like `mid * mid`, and costs accumulated over 10^5 elements. Exit early or cap at the target.
5. **Mixing templates.** Combining `while (lo <= hi)` with `lo = mid` gives infinite loops. Pick the invariant template and use it every time.
6. **Binary searching an unsorted array**, or using a comparator inconsistent with the sort order.
7. **Using `eps` termination for reals** instead of a fixed iteration count.
8. **Forgetting that $\log_2(10^{18}) \approx 60$.** An expensive check, say O(n log n) with n = 10^6, run 60 times may be too slow.

9. Practice problems

CSES

- 1620 — Factory Machines — https://cses.fi/problemset/task/1620
- 1085 — Array Division — https://cses.fi/problemset/task/1085

Codeforces

- 474B — Worms — https://codeforces.com/problemset/problem/474/B
- 706B — Interesting drink — https://codeforces.com/problemset/problem/706/B
- 1201C — Maximum Median — https://codeforces.com/problemset/problem/1201/C
- 371C — Hamburgers — https://codeforces.com/problemset/problem/371/C
- 760B — Frodo and pillows — https://codeforces.com/problemset/problem/760/B
- 1613C — Poisoned Dagger — https://codeforces.com/problemset/problem/1613/C

AtCoder

- ABC146 C — Buy an Integer — https://atcoder.jp/contests/abc146/tasks/abc146_c
- ABC077 C — Snuke Festival — https://atcoder.jp/contests/abc077/tasks/arc084_a

USACO

- 2016 December Silver — Counting Haybales — http://www.usaco.org/index.php?page=viewproblem2&cpid=666
- 2018 December Silver — Convention — http://www.usaco.org/index.php?page=viewproblem2&cpid=858

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Section 3.3 ("Binary search"): https://cses.fi/book/book.pdf
- TopCoder tutorial "Binary Search" by lovro, which covers the predicate/boundary view and the main-theorem formulation
- Codeforces EDU course "Binary Search" (steps 1–4), with lessons and practice problems: https://codeforces.com/edu/courses
- USACO Guide, Silver module "Binary Search": https://usaco.guide/silver/binary-search